Calcul Scientifique

SMP5

Rachid Moustabchir
Département de Physique

2025/10/25

Plan du cours

- Chapitre I: Python pour la Simulation et la Visualisation
- **Chapitre II: Équations Différentielles Ordinaires (EDO)**
- Chapitre III: Notions sur les systèmes dynamiques
- Chapitre IV: Résolution numérique des E.D.P.

Équations Différentielles Ordinaires (EDO)

- Une équation différentielle ordinaire (EDO) du premier ordre est une équation de la forme

$$\frac{dx}{dt} = f(x,t),$$

avec la condition initiale $x(t=0) = x_0$.

- Cela détermine la dépendance $x(t)$ pour $t > 0$.
- Dans de nombreuses applications physiques, t joue le rôle de la variable temps, bien que ce ne soit pas toujours le cas.

Quand avons-nous besoin de méthodes numériques pour les EDOs ?

- La solution d'une EDO

$$\frac{dx}{dt} = f(x,t), \quad x(t=0) = x_0$$

- peut être formellement écrite comme

$$x(t) = x_0 + \int_0^t f\,[x(t'), t']dt'.$$

- Si f ne dépend pas de x, la solution peut être obtenue par intégration (numérique).
- Dans d'autres cas, la solution peut être obtenue par séparation des variables.
- Dans tous les autres cas, la solution doit être obtenue numériquement.

Méthodes numériques pour les EDOs

- Typiquement, la solution est obtenue en effectuant de petits pas de $x(t)$ à $x(t+h)$.

Caractéristiques :

- **Explicite ou implicite**
  - **Méthodes explicites :** utilisent $x(t)$ pour calculer $x(t+h)$ directement :

    $$\mathbf{x(t + h) = F(x(t), t, h).}$$

  - **Méthodes implicites :** nécessitent de résoudre une équation pour $x(t+h)$:

    $$\mathbf{x(t + h) = F(x(t + h), x(t), t, h).}$$

- **Précision**
  - L'erreur de troncature à chaque pas est de l'ordre de $O(h^n)$.
  - Certains schémas sont explicitement réversibles dans le temps et/ou conservent l'énergie.
  - Les méthodes adaptatives ajustent la taille du pas h pour contrôler l'erreur selon la précision souhaitée.
- **Stabilité**
  - Si l'erreur accumulée est bornée.
- **Cohérence**
  - Les méthodes cohérentes reproduisent la solution exacte lorsque $h \to 0$.

Rappels Théorème de Cauchy-Lipschitz

- Soient $T > 0$, un intervalle $E \subset \mathbb{R}$ et $f : [0,T] \times E \to \mathbb{R}$ une fonction continue.
- On cherche les solutions de l'équation différentielle avec condition initiale (problème de Cauchy) :

$$(C) \quad \begin{cases} x' = f(t,x), & t \in]0,T] \\ x(0) = x_0, & x_0 \in \mathbb{R} \end{cases}$$

Théorème:

- S'il existe $L > 0$ tel que :

$$\forall t \in [0,T], \forall (x,z) \in E^2, \quad |f(t,x) - f(t,z)| \leq L|x - z|,$$

$\Longrightarrow$ alors pour tout $x_0 \in E$, le problème de Cauchy (C) admet une **solution unique**.

- Le Théorème de Cauchy-Lipschitz (ou Théorème de Picard-Lindelöf), en gros, dit :

"Si la fonction f est suffisamment régulière (techniquement, de classe C^1), alors pour **tout état initial** x_0 que nous choisissons, il existe une **unique solution** $x(t)$ qui part de cet état initial et qui satisfait l'équation différentielle. Cette solution existe pour un certain intervalle de temps."

Rappels Théorème de Cauchy-Lipschitz

- **Le Point Clé :** " f est de classe C^1 " C'est la condition magique qui active le théorème.
  - "**Classe C^1**" signifie que la fonction f **est continûment différentiable.**
  - Concrètement, cela veut dire que si nous calculons sa dérivée (c'est-à-dire comment f change lorsqu'on modifie légèrement x), cette dérivée existent et est elle-même une fonction continue.

Conclusion

- "Nous étudions un système régi par l'équation $x'(t) = f(x(t))$. Si la fonction f est très régulière (**de classe C^1**). Grâce au **théorème de Cauchy-Lipschitz**, cette régularité nous garantit que pour n'importe quel point de départ que l'on choisit, il y a **une et une seule** trajectoire possible pour le système. Il n'y a pas d'ambiguïté : le futur est (mathématiquement) unique et déterminé par le présent."
- C'est une information cruciale, car elle valide toute simulation numérique que nous pourrions faire. Si on lance une simulation à partir d'une condition initiale, nous pouvons être sûr que la solution que l'ordinateur calcule est la *bonne* (aux erreurs d'arrondi près), et non pas une solution parmi d'autres.

Schémas numériques

Principe

- On cherche à calculer une **approximation de** $t \mapsto x(t)$ solution de :

$$(C) \quad \begin{cases} x' = f(t,x), & t \in]0,T] \\ x(0) = x_0, & x_0 \in \mathbb{R} \end{cases}$$

pour des valeurs discrètes de t données par la suite $(t_k)_{k=0\ldots N}$, où $t_k = kh$ et $h = T/N$.

Première idée :

$$x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} x'(t)dt = x(t_k) + \int_{t_k}^{t_{k+1}} f(t, x(t))dt.$$

- Il suffit d'approcher l'intégrale !

Schémas numériques: Le schéma à un pas

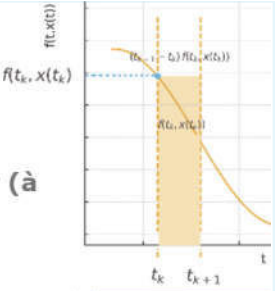
- On appelle **schéma à un pas** la construction de la suite récurrente (x_k) définie par :

$$x_{k+1} = x_k + h\varphi(t_k, x_k, h), \quad k \geq 0.$$

Exemples de schémas :

- Euler : $\varphi(t,x,h) = f(t,x)$
- Euler-Cauchy (ou Heun) : $\varphi(t,x,h) = \frac{1}{2}(f(t,x) + f(t+h, x+hf(t,x)))$
- Point milieu : $\varphi(t,x,h) = f\left(t+\frac{h}{2}, x+\frac{h}{2}f(t,x)\right)$

Schémas numériques: Le schéma d'Euler



- Si on approche l'intégrale par l'aire du **rectangle de hauteur (à gauche)** $f(t_k, x(t_k))$, on peut écrire :

$$\int_{t_k}^{t_{k+1}} f\big(t, x(t)\big)dt \approx (t_{k+1} - t_k).f\big(t_k, x(t_k)\big) + O(h^2),$$

$$\Rightarrow x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} f\big(t, x(t)\big)dt = x_k + h.f(t_k, x_k) + O(h^2), \quad k \geq 0.$$

- En négligeant les termes d'ordre supérieur en h, nous avons :

$$\begin{cases} x_{k+1} = x_k + hf(t_k, x_k) & \forall k \in [0, N-1] \\ x_0 = x(t_0) \end{cases}$$

Méthode d'Euler progressive

avec $x_k = x(t_k)$

- Nous pouvons appliquer cette relation de manière itérative à partir de $t = 0$ pour évaluer $x(t)$ pour $t > 0$.
- **Erreur :**
  - Locale (par pas de temps) : $O(h^2)$.
  - Globale ($N = t_{end}/h$) : $O(h)$.

Schémas numériques

Implémentation: Schéma d'Euler

Algorithme la function ode_euler_step.

Données :
f : Fonction de l'EDO (dx/dt = f(x, t))
x : Valeur actuelle de la variable dépendante (réel)
t : Valeur actuelle du temps (réel)
h : Pas de temps (réel > 0)
Résultat :
xnew : Nouvelle valeur de x après un pas d'Euler (réel)
1: Fonction xnew ← **ode_euler** (f, x, t, h):
2: xnew ← x + h * f(x, t)
3: Fin Fonction

$$x_{k+1} = x_k + hf(t_k, x_k)$$

Algorithme la fonction ode_euler.

Données :
f : Fonction de l'EDO (dx/dt = f(x, t))
x0 : Valeur initiale de x (réel)
t0 : Temps initial (réel)
h : Pas de temps (réel > 0)
nsteps : Nombre de pas (entier ≥ 0)
Résultats :
t : Tableau des temps $[t_0, t_1, ..., t_n]$
x : Tableau des valeurs de x $[x_0, x_1, ..., x_n]$
1: Fonction [t, x] ← **ode_euler** (f, x0, t0, h, nsteps):
2: t ← tableau de taille (nsteps + 1) initialisé à 0
3: x ← tableau de taille (nsteps + 1) initialisé à 0
4: t[0] ← t0
5: x[0] ← x0
6: Pour i allant de 0 à (nsteps - 1) faire
7: t[i + 1] ← t[i] + h
8: x[i + 1] ← ode_euler_step(f, x[i], t[i], h)
9: Fin Pour
10: Fin Fonction

Listing: (Python) la function ode_euler_step..

```
def ode_euler_step(f, x, t, h):
    """Effectuer un seul pas h en utilisant le schéma d'Euler.

    Arguments :
    f : la fonction qui définit l'EDO.
    x : la valeur de la variable dépendante à l'étape actuelle.
    t : la valeur actuelle de la variable de temps.
    h : le pas de temps

    Retour :
    xnew : la valeur de la variable dépendante à l'instant t + h
    """
    return x + h * f(x, t)
```

Listing: (Python) la fonction ode_euler.

```
def ode_euler(f, x0, t0, h, nsteps):
    """Résoudre une EDO dx/dt = f(x,t) de t = t0 à t = t0 + h*steps en
utilisant la méthode d'Euler.

    Arguments :
        f : la fonction qui définit l'EDO.
       x0 : la valeur initiale de la variable dépendante.
       t0 : la valeur initiale de la variable de temps.
        h : le pas de temps
   nsteps : le nombre total de pas d'Euler

    Retour :
    t, x : le couple de tableaux représentant le temps et la variable
dépendante
    """
    t = np.zeros(nsteps + 1)
    x = np.zeros(nsteps + 1)
    x[0] = x0
    t[0] = t0
    for i in range(0, nsteps):
        t[i + 1] = t[i] + h
        x[i + 1] = ode_euler_step(f, x[i], t[i], h)
    return t, x
```

Schémas numériques: Le schéma d'Euler

$$\frac{dx}{dt} = -x^3 + sint, \quad x(t=0) = 0.$$

- Résoudre l'équation à l'aide de la méthode d'Euler explicite pour différents **pas de temps**.

Listing: (Python) Example Extrait du script complet

```
import numpy as np

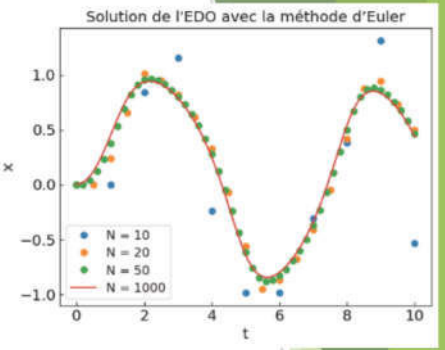
# Définir l'équation différentielle (EDO)
def f(x, t):
    return -x**3 + np.sin(t)

t0 = 0.
tend = 10.0
x0 = 0.

N = 10
h = (tend - t0) / N
ODEsolN10 = ode_euler(f, x0, t0, h, N)
N = 20
h = (tend - t0) / N
ODEsolN20 = ode_euler(f, x0, t0, h, N)
N = 50
.....

plt.plot(ODEsolN10[0], ODEsolN10[1], 'o', label='N = 10')
plt.plot(ODEsolN20[0], ODEsolN20[1], 'o', label='N = 20')
.....

plt.xlabel('t')
plt.ylabel('x')
plt.legend()
plt.title("Solution de l'EDO avec la méthode d'Euler")
plt.savefig("1.Euler.png", bbox_inches='tight') # Sauvegarde
plt.show() # Affichage
```



- **Pour les faibles valeurs de N :**
  - Les points calculés seront peu nombreux.
  - L'approximation sera grossière, avec des erreurs visibles.
- **Pour N = 1000** :
  - La solution sera beaucoup plus précise.
  - On obtiendra une courbe lisse représentant une bonne approximation de la solution réelle.

Schémas numériques: Méthodes de Runge-Kutta

- Les méthodes de Runge–Kutta sont des schémas à un pas pour approcher la solution de : $\begin{cases} y' = f(t, y), \ t \in]0, T] \\ y(t_0) = y_0, \quad y_0 \in \mathbb{R} \end{cases}$
- L'idée fondamentale des méthodes de Runge-Kutta est d'intégrer l'équation $y'(t) = f(t, y(t))$ sur $[t^n, t^{n+1}]$ et de calculer :

$$y(t^{n+1}) = y(t^n) + \int_{t^n}^{t^{n+1}} f(t, y(t))dt,$$

- en utilisant une formule d'intégration numérique à q points intermédiaires pour évaluer l'intégrale.

Carle Runge 1856–1927, mathématicien et physicien allemand



Martin Wilhelm Kutta 1867–1944, mathématicien allemand



John C. Butcher 1933, mathématicien appliqué néozélandais



Schémas numériques: Méthodes de Runge-Kutta

- Ce sont des méthodes à un pas :

$$y^{[n+1]} = y^{[n]} + h\Phi\left(t^n, y^{[n]}, h\right) \quad \text{avec}$$

$$\Phi(t, y, h) = \sum_{i=1}^{q} c_i \, k^{[i]}(t, y, h)$$

$$k^{[i]}(t, y, h) = f\left(t + ha_i, y + h\sum_{j=1}^{q} b_{i,j} \, k^{[j]}(t, y, h)\right), \quad 1 \le i \le q$$

- On peut représenter sous la forme d'un tableau dit **tableau de Butcher** :

a	B
	c^t

- avec $B = (b_{i,j})_{i,j\in[1,q]} \in M_q(\mathbb{R}), a = (a_i)_{i\in[1,q]} \in \mathbb{R}^q$ et $c = (c_i)_{i\in[1,q]} \in \mathbb{R}^q$

Schémas numériques: Méthodes de Runge-Kutta

Proposition (admis)

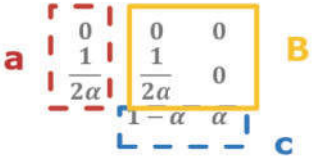
- Les méthodes de Runge-Kutta explicites sont stables si f est contractante en y (si la fonction $f(t,y)$ vérifie la condition de Lipschitz).
- Une méthode de Runge-Kutta est **d'ordre 0** si $a_i = \sum_{j=1}^{q} b_{ij}$.
- Une méthode de Runge-Kutta est **d'ordre 1** (et donc consistante) si elle est **d'ordre 0** et si $\sum_{i=1}^{q} c_i = 1$.
- Une méthode de Runge-Kutta est **d'ordre 2** si elle est **d'ordre 1** et si $\sum_{i=1}^{q} c_i a_i = \frac{1}{2}$.
- Une méthode de Runge-Kutta est **explicite** si la matrice B **est triangulaire** inférieure à diagonale nulle : $\forall (i,j) \in [1,q],\ j \geq i, \quad b_{ij} = 0$.

$$B = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 & 0 \\ b_{2,1} & 0 & 0 & \cdots & 0 & 0 \\ b_{3,1} & b_{3,2} & 0 & \cdots & 0 & 0 \\ b_{4,1} & b_{4,2} & b_{4,3} & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ b_{q,1} & b_{q,2} & b_{q,3} & \cdots & b_{q,q-1} & 0 \end{pmatrix}$$

Schémas numériques: Méthodes de Runge-Kutta

Runge-Kutta d'ordre 2:

- tableau de Butcher :



- $B \in M_q(\mathbb{R}), a \in \mathbb{R}^q$ **et** $c \in \mathbb{R}^q \Rightarrow q = 2$
- $k^{[i]}(t,y,h) = f\left(t + ha_i, y + h\sum_{j=1}^{q} b_{i,j}\, k^{[j]}(t,y,h)\right), \quad 1 \le i \le q$

$$\Rightarrow \begin{cases} k^{[1]}(t,y,h) = f(t,y) = f\left(t^n, y^{[n]}\right) \\ k^{[2]}(t,y,h) = f\left(t + \frac{h}{2\alpha}, y + \frac{h}{2\alpha} f(t,y)\right) = f\left(t^n + \frac{h}{2\alpha}, y^{[n]} + \frac{h}{2\alpha} f(t^n, y^{[n]})\right) \end{cases}$$

- $\Phi(t,y,h) = \sum_{i=1}^{q} c_i\, k^{[i]}(t,y,h)$

$$\Rightarrow \Phi(t,y,h) = (1-\alpha)k^{[1]} + \alpha k^{[2]}$$

Schémas numériques: Méthodes de Runge-Kutta

Runge-Kutta d'ordre 2:

- tableau de Butcher :

$$\begin{array}{ccc} 0 & 0 & 0 \\ \frac{1}{2\alpha} & \frac{1}{2\alpha} & 0 \\ & 1-\alpha & \alpha \end{array}$$

- $\Phi(t,y,h) = (1-\alpha)k^{[1]} + \alpha k^{[2]} = (1-\alpha)f(t,y) + \alpha f\left(t + \frac{h}{2\alpha}, y + \frac{h}{2\alpha} f(t,y)\right)$
- $y^{[n+1]} = y^{[n]} + h\Phi\left(t^n, y^{[n]}, h\right) = y^{[n]} + h(1-\alpha)k^{[1]} + h\alpha k^{[2]}$

$$\Rightarrow y^{[n+1]} = y^{[n]} + h\,(1-\alpha) f(t^n, y^{[n]}) + h\alpha f\left(t^n + \frac{h}{2\alpha}, y^{[n]} + \frac{h}{2\alpha} f(t^n, y^{[n]})\right)$$

- $\alpha = \frac{1}{2}$, **méthode de Heun :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{2} f(t^n, y^{[n]}) + \frac{h}{2} f\left(t^{n+1}, y^{[n]} + hf(t^n, y^{[n]})\right).$$

- $\alpha = 1$, **méthode d'Euler modifiée ou méthode du point milieu :**

$$y^{[n+1]} = y^{[n]} + hf\left(t^n + \frac{h}{2}, y^{[n]} + \frac{h}{2} f(t^n, y^{[n]})\right).$$

Schémas numériques: Méthodes de Runge-Kutta

Exercice

- La méthode de Heun est donnée par

$$y^{[n+1]} = y^{[n]} + \frac{h}{2} f(t^n, y^{[n]}) + \frac{h}{2} f\left(t^{n+1}, y^{[n]} + hf(t^n, y^{[n]})\right).$$

- **Q.1** *Écrire la fonction algorithmique* REDHEUNVEC *permettant de résoudre un problème de Cauchy vectoriel par la méthode de Heun en utilisant au plus 2N évaluations de* f.
- **Q.2** *Écrire un programme algorithmique permettant de retrouver numériquement l'ordre de cette méthode.*

Schémas numériques: Méthodes de Runge-Kutta

- **R.1** Le schéma de Heun peut s'écrire sous la forme :

$$y^{[n+1]} = y^{[n]} + \frac{h}{2}(k_1^n + k_2^n) \qquad \text{Avec } \begin{aligned} k_1^n &= f(t^n, y^{[n]}) \\ k_2^n &= f(t^{n+1}, y^{[n]} + hk_1^n) \end{aligned}$$

- L'algorithme de la fonction **REDHeunVec** s'écrit alors :

Fonction REDHeunVec : résolution d'un problème de Cauchy par le schéma de Heun

Données :

$f : [t^0, t^0 + T] \times \mathbb{R}^d \to \mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n) = t^{n-1}, \forall n \in [1, N+1]$
Y — matrice réelle de dimension $d \times (N+1)$, $Y(:, n) = y^{(n-1)}, \forall n \in [1, N+1]$

1: **Fonction** $[t, Y] \leftarrow$ REDHeunVec(f, t^0, T, y^0, N)
2: $t \leftarrow$ DisReg$(t^0, t^0 + T, N)$
3: $h \leftarrow (t^0 + T - t^0)/N$
4: $Y(:, 1) \leftarrow y^0$
5: **Pour** $n \leftarrow 1$ **à** N **faire**
6: $k_1 \leftarrow f(t(n), Y(:, n))$
7: $k_2 \leftarrow f(t(n+1), Y(:, n) + h \cdot k_1)$
8: $Y(:, n+1) \leftarrow Y(:, n) + (h/2) \cdot (k_1 + k_2)$
9: **Fin**
10: **Fin**

Schémas numériques: Méthodes de Runge-Kutta

- **R. 2** Il est possible de vérifier numériquement l'ordre du schéma de Heun. Pour cela, on choisit un problème de Cauchy dont la solution exacte est connue et on calcule, pour différentes valeurs de h (différentes valeurs de N), le maximum de l'erreur commise entre la solution exacte et la solution numérique donnée par le schéma de Heun :

$$E(h) = \max_{n\in[0,N]} \| y(t^n) - y^{[n]} \|_\infty$$

- On représente ensuite la fonction $E(h)$ en fonction de h. La méthode de Heun étant d'ordre 2, on a alors :

$$E(h) = \mathcal{O}(h^2) \approx Ch^2 \quad \text{quand } h \text{ est suffisamment petit}$$

- On utilise alors une échelle logarithmique pour représenter la courbe. En effet, on a :

$$\log E(h) \approx \log(Ch^2) = \log C + 2\log h$$

- En posant $\boldsymbol{X = logh}$ et $\boldsymbol{Y = \log E(h)}$, coordonnées en échelle logarithmique, on a :

$$\boldsymbol{Y \approx logC + 2X}$$

- qui est l'équation d'une droite. La pente de cette droite est donc l'ordre de la méthode.

Schémas numériques: Méthodes de Runge-Kutta

- **Algorithme**

Algorithme de vérification de l'ordre

1: $t^0 \leftarrow 0$, $T \leftarrow 4\pi$
2: $f:(t,z) \mapsto \cos(t)+1$, $y_{ex}: t \mapsto \sin(t)+t$
3: $y^0 \leftarrow y_{ex}(t^0)$
4: $LN \leftarrow 100:50:1000$
5: $nLN \leftarrow$ longueur(LN)
6: $H \leftarrow$ zeros(nLN) ▷ pour stocker les h
7: $E \leftarrow$ zeros(nLN) ▷ pour stocker les erreurs
8: **Pour** $k \leftarrow 1$ **à** nLN **faire**
9: $N \leftarrow LN(k)$
10: $[t,y] \leftarrow$ REDHeunVec(f, t^0, T, y^0, N)
11: $E(k) \leftarrow \max(|y - y_{ex}(t)|)$
12: $H(k) \leftarrow T/N$
13: **Fin**
14: ***PlotLogXY*** (H,E) ▷ Représentation graphique

- Ligne 4 : $LN \leftarrow 100:50:1000$

LN est un vecteur contenant des valeurs allant de 100 à 1000 avec un pas de 50

- Ligne 5 : $nLN \leftarrow$ longueur(LN)

calcule le nombre d'éléments dans le vecteur LN : $nLN = 19$

Schémas numériques: Méthodes de Runge-Kutta

- **R.1** Le schéma de Heun peut s'écrire sous la forme :

$$y^{[n+1]} = y^{[n]} + \frac{h}{2}(k_1^n + k_2^n) \qquad \text{Avec } \begin{aligned} k_1^n &= f(t^n, y^{[n]}) \\ k_2^n &= f(t^{n+1}, y^{[n]} + hk_1^n) \end{aligned}$$

- L'algorithme de la fonction **REDHeunVec** s'écrit alors :

Fonction REDHeunVec : résolution d'un problème de Cauchy par le schéma de Heun

Données :

$f : [t^0, t^0 + T] \times \mathbb{R}^d \to \mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)

t^0 — réel, temps initial

T — réel > 0

y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale

N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n) = t^{n-1}, \forall n \in [1, N+1]$

Y — matrice réelle de dimension $d \times (N+1)$, $Y(:, n) = y^{(n-1)}, \forall n \in [1, N+1]$

1: **Fonction** $[t, Y] \leftarrow$ REDHeunVec(f, t^0, T, y^0, N)
2: $t \leftarrow$ DisReg$(t^0, t^0 + T, N)$
3: $h \leftarrow (t^0 + T - t^0)/N$
4: $Y(:, 1) \leftarrow y^0$
5: **Pour** $n \leftarrow 1$ **à** N **faire**
6: $k_1 \leftarrow f(t(n), Y(:, n))$
7: $k_2 \leftarrow f(t(n+1), Y(:, n) + h \cdot k_1)$
8: $Y(:, n+1) \leftarrow Y(:, n) + (h/2) \cdot (k_1 + k_2)$
9: **Fin**
10: **Fin**

Schémas numériques: Méthodes de Runge-Kutta

Runge-Kutta d'ordre 4

- La méthode explicite la plus utilisée est donnée par le tableau de Butcher suivant

$$\begin{array}{cccccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

- Ce qui donne le schéma explicite de Runge-Kutta d'ordre 4 :

$$k_1^{[n]} = f(t^n, y^{[n]})$$

$$k_2^{[n]} = f\left(t^n + \frac{h}{2}, y^{[n]} + \frac{h}{2}k_1^{[n]}\right)$$

$$k_3^{[n]} = f\left(t^n + \frac{h}{2}, y^{[n]} + \frac{h}{2}k_2^{[n]}\right)$$

$$k_4^{[n]} = f\left(t^n + h, y^{[n]} + hk_3^{[n]}\right)$$

- $y^{[n+1]} = y^{[n]} + \frac{h}{6}\left(k_1^{[n]} + 2k_2^{[n]} + 2k_3^{[n]} + k_4^{[n]}\right)$.

Schémas numériques: Méthodes de Runge-Kutta

Runge-Kutta d'ordre 4

- **Exercice 6.2**
- La méthode de Runge-Kutta d'ordre 4 est donnée par

$$k_1^{[n]} = f(t^n, y[n])$$
$$k_2^{[n]} = f\left(t^n + \frac{h}{2}, y[n] + \frac{h}{2}k_1^{[n]}\right)$$
$$k_3^{[n]} = f\left(t^n + \frac{h}{2}, y[n] + \frac{h}{2}k_2^{[n]}\right)$$
$$k_4^{[n]} = f\left(t^n + h, y[n] + hk_3^{[n]}\right)$$

- $y[n+1] = y[n] + \frac{h}{6}\left(k_1^{[n]} + 2k_2^{[n]} + 2k_3^{[n]} + k_4^{[n]}\right)$.
- **Q.1** *Écrire une fonction algorithmique* REDRK4Vec *permettant de résoudre un problème de Cauchy vectoriel par la méthode de Runge-Kutta d'ordre 4.*
- **Q.2** *Écrire un programme algorithmique permettant de retrouver numériquement l'ordre de cette méthode.*

Schémas numériques: Méthodes de Runge-Kutta

- **R.1** Le schéma de Runge-Kutta d'ordre 4 est donnée par :

$$y[n+1] = y[n] + \frac{h}{6}\left(k_1^{[n]} + 2k_2^{[n]} + 2k_3^{[n]} + k_4^{[n]}\right).$$

Avec
$$k_1 \leftarrow k_1^{[n]}(t^n, y^{[n]})$$
$$k_2 \leftarrow k_2^{[n]}(t^n, y^{[n]})$$
$$k_3 \leftarrow k_3^{[n]}(t^n, y^{[n]})$$
$$k_4 \leftarrow k_4^{[n]}(t^n, y^{[n]})$$

- Il n'est pas nécessaire de stocker l'ensemble des vecteurs $k_1^{[n]}, \dots, k_4^{[n]}, n \in [1, N]$.
- Pour minimiser l'occupation mémoire, on calcule leurs valeurs à chaque itération.

Schémas numériques: Méthodes de Runge-Kutta

- **R.1** L'algorithme de la fonction **REDRK4Vec** s'écrit alors :

Fonction REDRK4Vec : résolution d'un problème de Cauchy par le schéma de RK4

Données :

$f : [t^0, t^0 + T] \times \mathbb{R}^d \to \mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n) = t^{n-1}, \forall n \in [1, N+1]$
Y — matrice réelle de dimension $d \times (N+1)$, $Y(:,n) = y^{(n-1)}, \forall n \in [1, N+1]$

1: **Fonction** $[t, Y] \leftarrow$ REDRK4Vec(f, t^0, T, y^0, N)
2: $t \leftarrow$ DisReg$(t^0, t^0 + T, N)$
3: $h \leftarrow (t^0 + T - t^0)/N$
4: $Y(:,1) \leftarrow y^0$
5: **Pour** $n \leftarrow 1$ **à** N **faire**
6: $k_1 \leftarrow f(t(n), Y(:,n))$
7: $k_2 \leftarrow f(t(n) + h/2, Y(:,n) + (h/2)k_1)$
8: $k_3 \leftarrow f(t(n) + h/2, Y(:,n) + (h/2)k_2)$
9: $k_4 \leftarrow f(t(n) + h, Y(:,n) + hk_3)$
10: $Y(:,n+1) \leftarrow Y(:,n) + (h/6) \cdot (k_1 + 2k_2 + 2k_3 + k_4)$
11: **Fin**
12: **Fin**

Schémas numériques: Méthodes de Runge-Kutta

- **R. 2** Il est possible de vérifier numériquement l'ordre du schéma de Heun. Pour cela, on choisit un problème de Cauchy dont la solution exacte est connue et on calcule, pour différentes valeurs de h (différentes valeurs de N), le maximum de l'erreur commise entre la solution exacte et la solution numérique donnée par le schéma de Heun :

$$E(h) = \max_{n\in[0,N]} \| y(t^n) - y^{[n]} \|_\infty$$

- On représente ensuite la fonction $E(h)$ en fonction de h. La méthode de Runge-Kutta 4 étant d'ordre 4, on a alors :

$$E(h) = \mathcal{O}(h^4) \approx Ch^4 \quad \text{quand } h \text{ est suffisamment petit}$$

- On utilise alors une échelle logarithmique pour représenter la courbe. En effet, on a :

$$\log E(h) \approx \log(Ch^4) = \log C + 4\log h$$

- En posant $\boldsymbol{X = logh}$ et $\boldsymbol{Y = \log E(h)}$, coordonnées en échelle logarithmique, on a :

$$\boldsymbol{Y \approx logC + 4X}$$

- qui est l'équation d'une droite. La pente de cette droite est donc l'ordre de la méthode.

Schémas numériques: Méthodes de Runge-Kutta

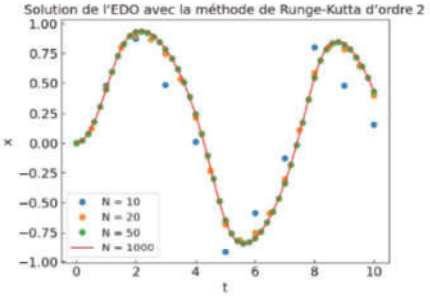
- **Algorithme**

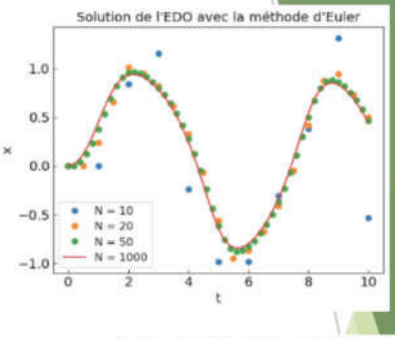
Algorithme de vérification de l'ordre

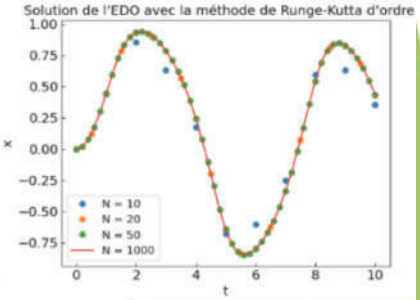
1: $t^0 \leftarrow 0$, $T \leftarrow 4\pi$
2: $f:(t,z) \mapsto \cos(t)+1$, $y_{ex}: t \mapsto \sin(t)+t$
3: $y^0 \leftarrow y_{ex}(t^0)$
4: $LN \leftarrow 100:50:1000$
5: $nLN \leftarrow$ longueur(LN)
6: $H \leftarrow$ zeros(nLN) ▷ pour stocker les h
7: $E \leftarrow$ zeros(nLN) ▷ pour stocker les erreurs
8: **Pour** $k \leftarrow 1$ **à** nLN **faire**
9: $N \leftarrow LN(k)$
10: $[t,y] \leftarrow$ *REDRK4Vec*(f, t^0, T, y^0, N)
11: $E(k) \leftarrow \max(|y - y_{ex}(t)|)$
12: $H(k) \leftarrow T/N$
13: **Fin**
14: ***PlotLogXY*** (H,E) ▷ Représentation graphique

Schémas numériques: Euler/RK2/RK4

$$\frac{dx}{dt} = -x^3 + sint, \quad x(t=0) = 0.$$







- Pour N=10 ou N=20, Euler diverge notablement de RK4.
- RK2 suit une trajectoire plus proche mais pas parfaitement superposée.
- RK4 donne une solution de référence.
- L'erreur relative diminue fortement en allant de Euler → RK2 → RK4.
- Avec N=1000, toutes les méthodes convergent numériquement.

Schémas numériques: Méthodes à pas multiples

Exemple : schéma de point milieu

- Considérons la méthode à deux pas définie par la récurrence:

$$y^{[n+1]} = y^{[n-1]} + 2hf(t^n, y^{[n]}).$$

- Cette méthode est d'ordre 2.
- Ces méthodes à pas multiples posent des problèmes de démarrage:
  - ici, on ne peut calculer directement $y^{[1]}$.
  - Les premières valeurs doivent être calculées par un autre schéma.

Schémas numériques: Méthodes à pas multiples

Méthodes explicites d'Adams-Bashforth

- On note en abrégé $f^{[n]} = f(t^n, y^{[n]})$. Voici trois schémas :
  - **Schéma explicite d'Adams-Bashforth d'ordre 2 (2 pas) :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{2}\left(3f^{[n]} - f^{[n-1]}\right)$$

  - **Schéma explicite d'Adams-Bashforth d'ordre 3 (3 pas) :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{12}\left(23f^{[n]} - 16f^{[n-1]} + 5f^{[n-2]}\right)$$

  - **Schéma explicite d'Adams-Bashforth d'ordre 4 (4 pas) :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{24}\left(55f^{[n]} - 59f^{[n-1]} + 37f^{[n-2]} - 9f^{[n-3]}\right)$$

- Ces schémas sont **explicites** et leur ordre correspond au nombre de pas.

Schémas numériques: Méthodes à pas multiples

Exercice

- La méthode de **Adams-Bashforth d'ordre 4 explicite** est donnée par :

$$y^{[n+1]} = y^{[n]} + \frac{h}{24}\left(55f^{[n]} - 59f^{[n-1]} + 37f^{[n-2]} - 9f^{[n-3]}\right),$$

- Avec $f^{[n]} = f(t^n, y^{[n]})$.
- Écrire la fonction algorithmique REDAB4Vec permettant de résoudre un problème de Cauchy vectoriel par cette méthode.

Schémas numériques: Méthodes à pas multiples

Exercice

- Soit $(t^{[n]})_{n=0}^{N}$ la discrétisation régulière de $[t^0, t^0 + T]$ avec $h = T/N$.
- On a donc $t^{[n]} = t^0 + nh$, $\forall n \in [0, N]$.
- On ne peut utiliser ce schéma à pas multiples que pour $n \geq 4$.
- On va alors utiliser un schéma à un pas d'ordre (au moins) 4 pour calculer les 4 premiers termes $y^{[0]}, y^{[1]}, y^{[2]}$ et $y^{[3]}$ nécessaire pour démarrer le schéma.
- On choisi par exemple la méthode de Runge-Kutta d'ordre 4 pour initialiser ces 4 termes.

Schémas numériques: Méthodes à pas multiples

Exercice

- Deux possibilités :
  - on réécrit le schéma de RK d'ordre 4 pour ces 4 premiers termes:
    - 1: $Y(:,1) \leftarrow y^0$
    - 2: Pour $n \leftarrow 1$ à 3 faire
    - 3: $k_1 \leftarrow f(t(n), Y(:,n))$
    - 4: $k_2 \leftarrow f(t(n) + h/2, Y(:,n) + (h/2)k_1)$
    - 5: $k_3 \leftarrow f(t(n) + h/2, Y(:,n) + (h/2)k_2)$
    - 6: $k_4 \leftarrow f(t(n) + h, Y(:,n) + hk_3)$
    - 7: $Y(:,n+1) \leftarrow Y(:,n) + (h/6) * (k_1 + 2k_2 + 2k_3 + k_4)$
  - on utilise la fonction REDRK4Vec avec ses paramètres d'entrées judicieusement choisis :
    - 1: $[t_{ini}, Y_{ini}] \leftarrow REDRK4Vec(f, t^0, t^0 + 3 * h, y^0, 3)$
    - 2: Pour $n \leftarrow 1$ à 4 faire
    - 3: $Y(:,n) \leftarrow Y_{ini}(:,n)$
    - 4: Fin

Schémas numériques: Méthodes à pas multiples

Exercice

- En choisissant cette dernière solution, l'algorithme de la fonction REDAB4Vec s'écrit alors :

Fonction REDAB4Vec : résolution d'un problème de Cauchy par le schéma d'Adams-Bashforth d'ordre 4

Données :

$f: [t^0, t^0 + T] \times \mathbb{R}^d \to \mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n) = t^{n-1}, \forall n \in [1, N+1]$
Y — matrice réelle de dimension $d \times (N+1)$, $Y(:,n) = y^{(n-1)}, \forall n \in [1, N+1]$

1: **Fonction** $[t, Y] \leftarrow$ REDAB4Vec (f, t^0, T, y^0, N)
2: $t \leftarrow$ DisReg$(t^0, t^0 + T, N)$
3: $h \leftarrow (t^0 + T - t^0)/N$
4: $[t_{ini}, Y_{ini}] \leftarrow$ REDRK4Vec$(f, t^0, t^0 + 3 * h, y^0, 3)$
5: **Pour** $n \leftarrow 1$ **à 4 faire**
6: $Y(:,n) \leftarrow Y_{ini}(:,n)$
7: **Fin**
8: **Pour** $n \leftarrow 4$ **à** N **faire**
9: $k_0 \leftarrow f(t(n), Y(:,n))$
10: $k_1 \leftarrow f(t(n-1), Y(:,n-1))$
11: $k_2 \leftarrow f(t(n-2), Y(:,n-2))$
12: $k_3 \leftarrow f(t(n-3), Y(:,n-3))$
13: $Y(:,n+1) \leftarrow Y(:,n) + (h/24) * (55 * k_0 - 59 * k_1 + 37 * k_2 - 9 * k_3)$
15: **Fin**
16: **Fin**

Schémas numériques: Méthodes à pas multiples

Exercice

- Une autre version minimisant le nombre d'appels à la fonction f est donnée par :

8: $k_1 \leftarrow f(t(3), Y(:,3))$
9: $k_2 \leftarrow f(t(2), Y(:,2))$
10: $k_3 \leftarrow f(t(1), Y(:,1))$
11: **Pour** $n \leftarrow 4$ **à** N **faire**
12: $k_0 \leftarrow f(t(n), Y(:,n))$
13: $Y(:,n+1) \leftarrow Y(:,n) + (h/24) * (55 * k_0 - 59 * k_1 + 37 * k_2 - 9 * k_3)$
14: $k_3 \leftarrow k_2, k_2 \leftarrow k_1, k_1 \leftarrow k_0$
15: **Fin**

Schémas numériques: Méthodes à pas multiples

Exercice

- En choisissant cette dernière solution, l'algorithme de la fonction REDAB4Vec s'écrit alors :

Fonction REDAB4Vec : résolution d'un problème de Cauchy par le schéma d'Adams-Bashforth d'ordre 4

Données :

$f: [t^0, t^0 + T] \times \mathbb{R}^d \to \mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n) = t^{n-1}, \forall n \in [1, N+1]$
Y — matrice réelle de dimension $d \times (N+1)$, $Y(:,n) = y^{(n-1)}, \forall n \in [1, N+1]$

1: **Fonction** $[t, Y] \leftarrow$ REDAB4Vec (f, t^0, T, y^0, N)
2: $t \leftarrow$ DisReg$(t^0, t^0 + T, N)$
3: $h \leftarrow (t^0 + T - t^0)/N$
4: $[t_{ini}, Y_{ini}] \leftarrow$ REDRK4Vec$(f, t^0, t^0 + 3 * h, y^0, 3)$
5: **Pour** $n \leftarrow 1$ **à 4 faire**
6: $Y(:,n) \leftarrow Y_{ini}(:,n)$
7: **Fin**
8: $k_1 \leftarrow f(t(3), Y(:,3))$
9: $k_2 \leftarrow f(t(2), Y(:,2))$
10: $k_3 \leftarrow f(t(1), Y(:,1))$
11: **Pour** $n \leftarrow 4$ **à** N **faire**
12: $k_0 \leftarrow f(t(n), Y(:,n))$
13: $Y(:,n+1) \leftarrow Y(:,n) + (h/24) * (55 * k_0 - 59 * k_1 + 37 * k_2 - 9 * k_3)$
14: $k_3 \leftarrow k_2, k_2 \leftarrow k_1, k_1 \leftarrow k_0$
15: **Fin**
16: **Fin**

Schémas numériques: Méthodes à pas multiples

Méthodes implicites d'Adams-Moulton

- On note en abrégé $f^{[n]} = f(t^n, y^{[n]})$. Voici trois schémas :
  - **schéma d'Adams-Moulton d'ordre 2 à 1 pas :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{2}\left(f^{[n+1]} + f^{[n]}\right).$$

  - **schéma d'Adams-Moulton d'ordre 3 à 2 pas :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{12}\left(5f^{[n+1]} + 8f^{[n]} - f^{[n-1]}\right)$$

  - **schéma d'Adams-Moulton d'ordre 4 à 3 pas :**

$$y^{[n+1]} = y^{[n]} + \frac{h}{24}\left(9f^{[n+1]} + 19f^{[n]} - 5f^{[n-1]} + f^{[n-2]}\right)$$

Schémas numériques: Méthodes à pas multiples

Schéma prédicteur-correcteur

- Il s'agit là d'une des méthodes les plus employées.
- Une méthode de prédiction-correction procède en deux étapes à chacune des itérations :
  - **Prédiction** : on calcule une approximation de $\mathbf{y}(t_{n+1})$ notée $\overline{\mathbf{y}}^{[n+1]}$ à l'aide du **schéma explicite**
  - **Correction** : on utilise le schéma implicite dans lequel les fonctions $\mathbf{f}$ utilisant $\mathbf{y}^{[n+1]}$ sont remplacées par les fonctions $\mathbf{f}$ utilisant $\overline{\mathbf{y}}^{[n+1]}$.

l'algorithme

1: **Pour** $n \leftarrow 0$ **à** N **faire**
2: $\overline{\mathbf{y}}^{[n+1]} \leftarrow$ donné par un **schéma explicite**
3: $\mathbf{y}^{[n+1]} \leftarrow$ donné par **un schéma implicite**, inconnue $\mathbf{y}^{[n+1]}$ remplacée par $\overline{\mathbf{y}}^{[n+1]}$

Schémas numériques: Méthodes à pas multiples

Exemple

- Choisissons la méthode d'Euler explicite pour prédicteur et la méthode implicite des trapèzes comme correcteur.
- **Euler explicite :** $\mathbf{y}^{[n+1]} = \mathbf{y}^{[n]} + h\mathbf{f}\left(t^n, \mathbf{y}^{[n]}\right)$
- **Trapèze implicite :** $\mathbf{y}^{[n+1]} = \mathbf{y}^{[n]} + \frac{h}{2}(\mathbf{f}(t^n, \mathbf{y}^{[n]}) + \mathbf{f}(t^{n+1}, \mathbf{y}^{[n+1]}))$
- On obtient :

$$\begin{cases} \bar{y}^{[n+1]} = y^{[n]} + h\, f\left(t^n, y^{[n]}\right) & \textbf{(Prédiction)} \\ y^{[n+1]} = y^{[n]} + \dfrac{h}{2}\left(f(t^n, y^{[n]}) + f(t^{n+1}, \bar{y}^{[n+1]})\right) & \textbf{(Correction)} \end{cases}$$

- **Remarque** *On retrouve ici, pour ce cas simple, une formule de Runge-Kutta 2.*
- En pratique, on peut utiliser un schéma d'Adams explicite pour la prédiction et un autre implicite pour la correction.

Schémas numériques: Méthodes à pas multiples

Exercice

- On pose $\mathbf{f}^{[n]} = \mathbf{f}(t^n, \mathbf{y}^{[n]})$. La **méthode de Adams-Bashforth d'ordre 4** explicite est donnée par

$$\mathbf{y}^{[n+1]} = \mathbf{y}^{[n]} + \frac{h}{24}\left(55\mathbf{f}^{[n]} - 59\mathbf{f}^{[n-1]} + 37\mathbf{f}^{[n-2]} - 9\mathbf{f}^{[n-3]}\right)$$

- et la **méthode de Adams-Moulton d'ordre 4** implicite par

$$\mathbf{y}^{[n+1]} = \mathbf{y}^{[n]} + \frac{h}{24}\left(9\mathbf{f}^{[n+1]} + 19\mathbf{f}^{[n]} - 5\mathbf{f}^{[n-1]} + \mathbf{f}^{[n-2]}\right)$$

- avec $\mathbf{f}^{[n]} = \mathbf{f}(t^n, \mathbf{y}^{[n]})$.
- *Écrire la fonction algorithmique* REDPC4Vec *permettant de résoudre un problème de Cauchy (vectoriel) par une méthode de prédiction-correction utilisant ces deux schémas. On minimisera le nombre d'appel à la fonction* $\mathbf{f}$ *dans la boucle principale.*

Schémas numériques: Méthodes à pas multiples

Exercice

- On utilise le schéma de Runge-Kutta d'ordre 4 pour initialiser les 4 premières valeurs.
- Ensuite on utilise comme prédicteur le schéma explicite et comme correcteur le schéma implicite.
- Le principe est donc
  - Calcul à l'aide du prédicteur :

$$\hat{y}^{[n+1]} = y^{[n]} + \frac{h}{24}\left(55f^{[n]} - 59f^{[n-1]} + 37f^{[n-2]} - 9f^{[n-3]}\right)$$

  - Calcul à l'aide du correcteur :

$$\hat{f}^{[n+1]} = f(t^{n+1}, \hat{y}^{[n+1]})\ y^{[n+1]} = y^{[n]} + \frac{h}{24}\left(9\hat{f}^{[n+1]} + 19f^{[n]} - 5f^{[n-1]} + f^{[n-2]}\right)$$

- L'algorithme de la fonction REDPC4Vec s'écrit alors :

Schémas numériques: Méthodes à pas multiples

Exercice

Fonction REDPC4Vec : résolution d'un problème de Cauchy par prédiction-correction

Données :

$f:[t^0,t^0+T]\times\mathbb{R}^d\to\mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n)=t^{n-1},\forall n\in[1,N+1]$
Y — matrice réelle de dimension $d\times(N+1)$, $Y(:,n)=y^{(n-1)},\forall n\in[1,N+1]$

1: **Fonction** $[t,Y]\leftarrow$ REDPC4Vec(f,t^0,T,y^0,N)
2: $t\leftarrow$ DisReg(t^0,t^0+T,N)
3: $h\leftarrow(t^0+T-t^0)/N$
4: $[t_{ini},Y_{ini}]\leftarrow$ REDRK4Vec$(f,t^0,t^0+3*h,y^0,3)$
5: **Pour** $n\leftarrow 1$ **à** 4 **faire**
6: $Y(:,n)\leftarrow Y_{ini}(:,n)$
7: **Fin**
8: **Pour** $n\leftarrow 4$ **à** N **faire**
9: $k_0\leftarrow f(t(n),Y(:,n))$
10: $k_1\leftarrow f(t(n-1),Y(:,n-1))$
11: $k_2\leftarrow f(t(n-2),Y(:,n-2))$
12: $k_3\leftarrow f(t(n-3),Y(:,n-3))$
13: $\tilde{Y}\leftarrow Y(:,n)+(h/24)*(55*k_0-59*k_1+37*k_2-9*k_3)$
14: $\tilde{F}\leftarrow f(t(n+1),\tilde{Y})$
15: $Y(:,n+1)\leftarrow Y(:,n)+(h/24)*(9*\tilde{F}+19*k_0-5*k_1+k_2)$
19: **Fin**
20: **Fin**

Schémas numériques: Méthodes à pas multiples

Exercice

- Une autre version minimisant le nombre d'appels à la fonction f est donnée par :

8: $k_1 \leftarrow f(t(3), Y(:,3))$

9: $k_2 \leftarrow f(t(2), Y(:,2))$

10: $k_3 \leftarrow f(t(1), Y(:,1))$

11: **Pour** $n \leftarrow 4$ **à** N **faire**

12: $k_0 \leftarrow f(t(n), Y(:,n))$

13: $\hat{Y} \leftarrow Y(:,n) + (h/24) * (55 * k_0 - 59 * k_1 + 37 * k_2 - 9 * k_3)$

14: $\hat{F} \leftarrow f(t(n+1), \hat{Y})$

15: $Y(:,n+1) \leftarrow Y(:,n) + (h/24) * (9 * \hat{F} + 19 * k_0 - 5 * k_1 + k_2)$

16: $k_3 \leftarrow k_2$

17: $k_2 \leftarrow k_1$

18: $k_1 \leftarrow k_0$

19: **Fin**

Schémas numériques: Méthodes à pas multiples

Exercice

Fonction REDPC4Vec : résolution d'un problème de Cauchy par prédiction-correction

Données :

$f:[t^0,t^0+T]\times\mathbb{R}^d\to\mathbb{R}^d$ — fonction d'un problème de Cauchy (scalaire ou vectoriel)
t^0 — réel, temps initial
T — réel > 0
y^0 — un vecteur de $\mathbb{R}^d$, donnée initiale
N — un entier non nul (nombre de pas de discrétisation)

Résultats :

t — vecteur de $\mathbb{R}^{N+1}$, $t(n)=t^{n-1},\forall n\in[1,N+1]$
Y — matrice réelle de dimension $d\times(N+1)$, $Y(:,n)=y^{(n-1)},\forall n\in[1,N+1]$

1: **Fonction** $[t,Y]\leftarrow$ REDPC4Vec(f,t^0,T,y^0,N)
2: $t\leftarrow$ DisReg(t^0,t^0+T,N)
3: $h\leftarrow(t^0+T-t^0)/N$
4: $[t_{ini},Y_{ini}]\leftarrow$ REDRK4Vec$(f,t^0,t^0+3*h,y^0,3)$
5: **Pour** $n\leftarrow 1$ **à** 4 **faire**
6: $Y(:,n)\leftarrow Y_{ini}(:,n)$
7: **Fin**
8: $k_1\leftarrow f(t(3),Y(:,3))$
9: $k_2\leftarrow f(t(2),Y(:,2))$
10: $k_3\leftarrow f(t(1),Y(:,1))$
11: **Pour** $n\leftarrow 4$ **à** N **faire**
12: $k_0\leftarrow f(t(n),Y(:,n))$
13: $\hat{Y}\leftarrow Y(:,n)+(h/24)*(55*k_0-59*k_1+37*k_2-9*k_3)$
14: $\hat{F}\leftarrow f(t(n+1),\hat{Y})$
15: $Y(:,n+1)\leftarrow Y(:,n)+(h/24)*(9*\hat{F}+19*k_0-5*k_1+k_2)$
16: $k_3\leftarrow k_2$
17: $k_2\leftarrow k_1$
18: $k_1\leftarrow k_0$
19: **Fin**
20: **Fin**

Schémas numériques: Méthodes à pas multiples

Comparaison avec Runge-Kutta

- L'intérêt d'une méthode de résolution numérique d'équations différentielles se mesure principalement suivant deux critères:
  - son coût pour obtenir une précision donnée (c'est à dire le nombre d'évaluations de fonctions par étapes).
  - sa stabilité.
- La caractéristique des méthodes de Runge-Kutta est que le pas est assez facile à adapter, la mise en oeuvre informatique plus aisée.
- Mais pour des méthodes du même ordre de consistance, les méthodes de Runge-Kutta exigent plus d'évaluations de fonctions.
- Quand on sait que la solution de l'équation n'est pas sujette à de brusques variations de la dérivée, on prendra une méthode de type prédicteur-correcteur mais, si le pas doit être adapté plus précisément, on préférera une méthode de Runge-Kutta.

Pas de temps adaptatif

- Le choix du pas de temps h est crucial pour atteindre la précision souhaitée tout en optimisant les ressources.
  - h **trop grand** : la précision désirée n'est pas atteinte.
  - h **trop petit** : gaspillage de ressources calculatoires.
- Le **pas de temps adaptatif** consiste à **faire varier dynamiquement le pas de temps h** pendant la résolution d'une EDO, **en fonction du comportement de la solution**.
- Contrairement aux méthodes à **pas fixe**, le pas adaptatif permet de :
  - **réduire le pas h** lorsque la solution présente de fortes variations (zones raides, oscillations, non-linéarités marquées),
  - **augmenter le pas h** lorsque la solution est varie lentement (zones stables).

Méthodes d'estimation d'erreur :

- Comparer deux pas h avec un pas $2h$.
- Utiliser deux méthodes d'ordres différents (ex. Runge-Kutta-Fehlberg RKF45).

Pas adaptatif dans RK4

Idée générale du pas adaptatif

- On ajuste dynamiquement le pas de temps h pendant l'intégration pour maintenir **l'erreur locale contrôlée** (inférieure à une tolérance δ).
- La méthode RK4 a une **erreur locale** (par pas) de la forme :

vraie solution = approximation + erreur

$$\Rightarrow x(t+h) = x_{\mathbf{RK4}}(t+h) + ch^5 + o(h^5)$$

  - c est une **constante** dépendant de $f(t,x)$ et de ses dérivées.
  - h est le **pas de temps**.
  - $o(h^5)$ représente des termes d'ordre supérieur (négligeables).
- donc l'**erreur locale par étape** (i.e., l'erreur commise à chaque pas) est : $\varepsilon(h) \approx ch^5$
- Donc :
  - Pour **un pas de taille** h : erreur $\sim ch^5$
  - Pour **un pas de taille** $2h$: erreur $\sim c(2h)^5 = 32ch^5$

Pas adaptatif dans RK4 via un double pas

Double pas pour estimer l'erreur

- On veut approximer $x(t+2h)$ par deux méthodes :

Deux petits pas RK4 de taille h :

- On note cette approximation x_1
- vraie solution : $\boldsymbol{x(t+2h) \approx x_1 + 2ch^5}$

Un seul grand pas RK4 de taille $2h$:

- On note cette approximation x_2
- vraie solution : $\boldsymbol{x(t+2h) \approx x_2 + c(2h)^5 = x_2 + 32ch^5}$

$$\Rightarrow x_1 + 2ch^5 \approx x_2 + 32ch^5 \Rightarrow x_1 - x_2 \approx 30ch^5 \Rightarrow ch^5 \approx \frac{|x_1 - x_2|}{30}$$

$$\Rightarrow \boldsymbol{\varepsilon_{RK4} \approx |ch^5| = \frac{|x_1 - x_2|}{30}}$$

- Cette estimation nous permet de **contrôler l'erreur** localement.

Pas adaptatif dans RK4 via un double pas

Condition pour que l'erreur locale soit contrôlée

- Choisir un nouveau pas h' tel que l'erreur associée à ce pas vérifie une condition de tolérance locale δ par unité de temps :

$$|c(h')^5| \leq h' \cdot \delta$$

Calcul du nouveau pas optimal

- Puisque $|ch^5| \approx \frac{|x_1 - x_2|}{30} \Rightarrow |c| = \frac{|x_1 - x_2|}{30h^5} \Rightarrow |c(h')^5| = \frac{|x_1 - x_2|}{30h^5} \cdot (h')^5$
- On impose maintenant la condition: $|c(h')^5| = \frac{|x_1 - x_2|}{30h^5} \cdot (h')^5 \leq h' \cdot \delta$

$$\Rightarrow (h')^4 \leq \frac{30h^5\delta}{|x_1 - x_2|} \Rightarrow h' \leq \left(\frac{30h^5\delta}{|x_1 - x_2|}\right)^{1/4}$$

- Comme on part d'un pas h, on réécrit la formule sous la forme :

$$h' = h \cdot \left(\frac{30h\delta}{|x_1 - x_2|}\right)^{1/4}$$

- Ce qui est la formule classique du **pas adaptatif** pour RK4.

Pas adaptatif dans RK4 via un double pas

Règle d'adaptation du pas

- Le nouveau pas optimal: $h' = h \cdot \left(\frac{30h\delta}{|x_1 - x_2|}\right)^{1/4}$
- Cette formule permet de réduire ou augmenter le pas en fonction de l'erreur. On répète le calcul jusqu'à respecter la δ.
- **Cas 1** $h' > h$: L'erreur estimée est plus petite que la tolérance ⇒le pas utilisé était trop petit.
  - **Action :** On accepte la valeur $x(t + 2h)$, On augmente le pas : $h \to h'$, et on continue vers $t \to t + 2h$
  - **Gain de performance :** on prend moins de pas ⇒ calcul plus rapide.
- **Cas 2** $h' < h$: L'erreur est plus grande que la tolérance → le pas utilisé est trop grand.
  - **Action :** On rejette la valeur calculée $x(t + 2h)$, on réduit le pas :$h \to h'$ et on refait les calculs à partir de $x(t)$ avec le nouveau h
  - **But :** respecter la précision voulue sans dépasser l'erreur admissible δ.

Pas adaptatif dans RK4 via un double pas

Implémentation:

Listing: (Python) la méthode de RK4.

```
# Schéma de Runge-Kutta d'ordre 4 (RK4)
def ode_rk4_step(f, x, t, h):
    """Effectue un seul pas h avec la méthode de Runge-Kutta
d'ordre 4.

    Arguments :
    f : fonction définissant l'EDO.
    x : valeur actuelle de la variable dépendante.
    t : instant actuel.
    h : pas de temps.

    Retour :
    xnouveau : estimation de x à l'instant t + h.
    """
    k1 = h * f(x, t)
    k2 = h * f(x + k1 / 2., t + h / 2.)
    k3 = h * f(x + k2 / 2., t + h / 2.)
    k4 = h * f(x + k3, t + h)
    return x + (k1 + 2. * k2 + 2. * k3 + k4) / 6.
```

Listing: (Python) RK4 avec pas adaptatif

```
def ode_rk4_adaptive(f, x0, t0, h0, tmax, delta=1.e-6):

    ts = [t0]
    xs = [x0]
    h = h0
    t = t0
    i = 0
    while (t < tmax):
        if (t + h >= tmax):
            ts.append(tmax)
            h = tmax - t
            xs.append(ode_rk4_step(f, xs[i], ts[i], h))
            t = tmax
            break

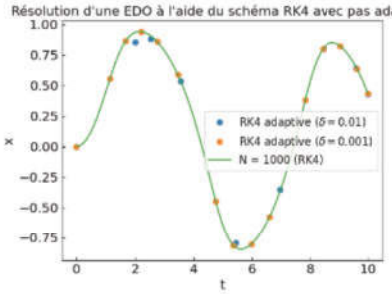
        x1 = ode_rk4_step(f, xs[i], ts[i], h)
        x1 = ode_rk4_step(f, x1, ts[i] + h, h)
        x2 = ode_rk4_step(f, xs[i], ts[i], 2 * h)

        rho = 30. * h * delta / np.abs(x1 - x2)
        if rho < 1.:
            h *= rho ** (1 / 4.)
        else:
            if (t + 2. * h) < tmax:
                xs.append(x1)
                ts.append(t + 2 * h)
                t += 2 * h
            else:
                xs.append(ode_rk4_step(f, xs[i], ts[i], h))
                ts.append(t + h)
                t += h
            i += 1
            h = min(2. * h, h * rho ** (1 / 4.))

    return ts, xs
```

Pas adaptatif dans RK4 via un double pas

$$\frac{dx}{dt} = -x^3 + sint, \quad x(t=0) = 0.$$



- Les points avec la courbe se superposent globalement.
- Les méthodes adaptatives ont moins de points (N=10).
- Le pas adaptatif permet de conserver la précision dans les zones critiques tout en évitant de faire trop de calculs inutiles dans les zones plates.
- Cela démontre l'efficacité des schémas adaptatifs en résolution d'EDO.

Schémas numériques: Convergence, approche expérimentale

- L'**erreur maximale** est définie comme :

$$E(h) = \max_{0 \le k \le N} |x(t_k) - x_k|$$

où :

- $x(t_k)$: valeur exacte au temps t_k
- x_k : valeur numérique approchée au temps t_k
- h : pas de temps utilisé ($h = \frac{T}{N}$)
- N : nombre d'itérations (ou de pas)

- On dit que le schéma à un pas : $x_{k+1} = x_k + h\varphi(t_k, x_k, h), \quad k \ge 0.$

est **convergent** si :

$$\lim_{h \to 0} E(h) = 0$$

Schémas numériques: Convergence, approche expérimentale

- Etudier expérimentalement la convergence des schémas d'Euler et d'Euler-Cauchy sur: $\frac{dx}{dt} = -tx + t, \quad x(0) = 0.$
- La solution exacte est : $x(t) = 1 - \exp(-t^2/2)$

Listing: (Python) Example Extrait du script complet

```
# Définir l'équation différentielle (EDO) et la solution exacte:
def f(x, t):
    return -t * x + t

def sol_exacte(t):
    return 1 - np.exp(-t**2 / 2)

# Étude de convergence : comparer erreurs max pour différents pas h
def test_convergence():
    h_values = [0.2, 0.1, 0.05, 0.025, 0.0125]
    erreurs_euler = []
    erreurs_cauchy = []

    t0 = 0.0
    tend = 2.0
    x0 = 0.0
for h in h_values:
        N = int((tend - t0) / h)

        t_euler, x_euler = ode_euler(f, x0, t0, h, N)
        t_cauchy, x_cauchy = ode_euler_cauchy(f, x0, t0, h, N)
        x_ref = sol_exacte(t_euler)
```

Listing: (Python) Example Extrait du script complet (Suite)

```
        err_euler = np.max(np.abs(x_ref - x_euler))
        err_cauchy = np.max(np.abs(x_ref - x_cauchy))

        erreurs_euler.append(err_euler)
        erreurs_cauchy.append(err_cauchy)

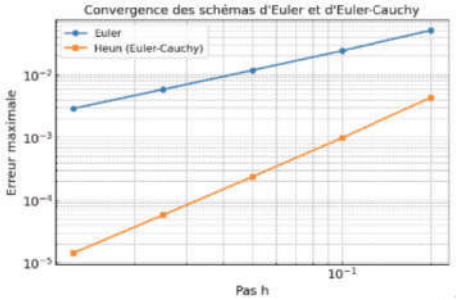
# Affichage en échelle log-log
    plt.figure(figsize=(8, 5))
    plt.loglog(h_values, erreurs_euler, 'o-', label='Euler', linewidth=2)
    plt.loglog(h_values, erreurs_cauchy, 's-', label='Heun (Euler-
Cauchy)', linewidth=2)
    plt.xlabel('Pas de temps h')
    plt.ylabel('Erreur maximale')
    plt.title("Convergence des schémas d'Euler et d'Euler-Cauchy")
    plt.grid(True, which='both', ls='--')
    plt.legend()
    plt.savefig("3.Convergence_Euler_vs_Heun.png",
bbox_inches='tight')
    plt.show()

# Exécution
test_convergence()
```

Schémas numériques: Convergence, approche expérimentale

$$\frac{dx}{dt} = -tx + t, \quad x(0) = 0.$$

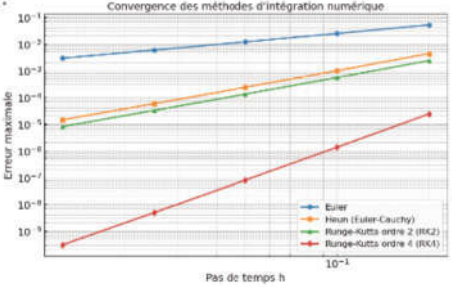
- Le **graphique affiche l'erreur maximale en fonction du pas** h.
  - L'axe des x est **log(h)**, et l'axe des y est **log($E(h)$)**.
- Cela permet de visualiser **le taux de convergence** :
  - Euler doit avoir une pente proche de 1 (erreur $\propto h$)
  - Heun doit avoir une pente proche de 2 (erreur $\propto h^2$)
- Plus les courbes sont **droites** sur un graphique log-log, plus cela confirme que le schéma est **convergent** avec un ordre donné.



Schémas numériques: Convergence, approche expérimentale

$$\frac{dx}{dt} = -tx + t, \quad x(0) = 0.$$

- Euler a une pente proche de 1 (erreur $\propto h$)
- Heun et RK2 ont une pente proche de 2 (erreur $\propto h^2$)
- RK4 a une pente proche de 4 (erreur $\propto h^4$)
- Plus la pente est élevée, plus la méthode est rapide à converger lorsque h diminue.
- Cela met en évidence les performances supérieures des méthodes de Runge-Kutta, notamment RK4, qui atteint une très bonne précision même avec un pas relativement grand.



Schémas numériques: Convergence, approche expérimentale

$$\frac{dx}{dt} = -x^3 + sint, \quad x(t=0) = 0.$$

- Equation non linéaire à cause du terme x^3, et elle ne possède pas de solution analytique connue.
  - Nous ne pouvons pas comparer directement nos résultats numériques à une expression exacte de la solution.

Stratégie adoptée

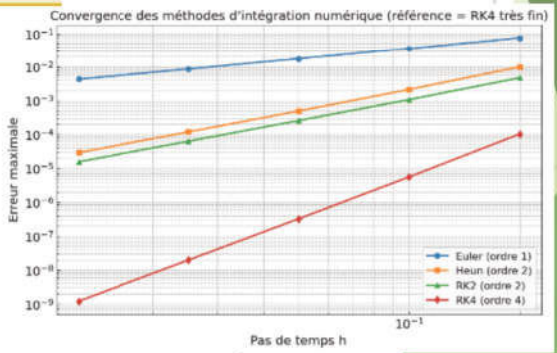
- Pour évaluer la précision des méthodes numériques, On adopte une solution de référence numérique très précise.
- Cette solution est obtenue à l'aide de la méthode de RK4 avec un pas de temps très fin (par exemple $h = 10^{-4}$).
- Ce type d'intégration fournit une approximation très fidèle de la solution exacte, suffisamment précise pour servir de référence lors de l'évaluation des erreurs.

Schémas numériques: Convergence, approche expérimentale

$$\frac{dx}{dt} = -x^3 + sint, \quad x(t=0) = 0.$$

Listing: (Python) solution de référence obtenue à l'aide de la méthode de RK4

```
# Solution de référence très précise par RK4 car l'équation n'admet
pas de solution exacte connue
def reference_solution(f, x0, t0, tend, h_ref):
    N = int((tend - t0) / h_ref)
    return ode_rk4(f, x0, t0, h_ref, N)
```



Schémas numériques: Convergence, la théorie : la consistance

- La convergence d'un schéma résulte de deux propriétés complémentaires : **la consistance** et **la stabilité**.
- Soit : $e_k = x(t_{k+1}) - x(t_k) - h\varphi(t_k, x(t_k), h)$,
- On dit que le schéma est **consistant** si :

$$\lim_{h \to 0} \sum_{k=0}^{N-1} |e_k| = 0.$$

- S'il existe $C_1 > 0$ tel que : $|e_k| < C_1 h^{p+1}$, on dit que le schéma est d'**ordre p**.
  - Le schéma d'Euler est **d'ordre 1**,
  - Le schéma d'Euler-Cauchy est **d'ordre 2**.

Schémas numériques: Convergence, la théorie : la stabilité

- Soient les deux suites (x_k) et $(\tilde{x}_k)$ définies par $x_0, \tilde{x}_0$ et, pour $k \geq 0$:

$$\begin{cases} x_{k+1} = x_k + \varphi(t_k, x_k, h), \\ \tilde{x}_{k+1} = \tilde{x}_k + \varphi(t_k, \tilde{x}_k, h) + \varepsilon_k, \end{cases}$$

- On dit que le schéma est **stable** s'il existe $S > 0$ tel que :

$$\max_{k=0\ldots N} |\tilde{x}_k - x_k| \leq S \left(|\tilde{x}_0 - x_0| + \sum_{k=0}^{N-1} |\varepsilon_k| \right)$$

- Le calcul de la constante S fait intervenir :
  - la formule du schéma,
  - le temps final T,
  - et la constante de Lipschitz L de f.
- Si f est lipschitzienne, les schémas d'Euler et d'Euler-Cauchy sont **stables**.

Schémas numériques: Convergence, la théorie

Théorème

- Si le schéma est **stable**, **consistant** et d'**ordre** p, alors il est **convergent** et il existe C_2 tel que :

$$\max_{k=0\ldots N} |x_k - x(t_k)| \leq C_2 h^p$$

Quelques remarques :

- La constante C_2 dépend du schéma, de T et des propriétés de la solution x (donc de f) :
- Le choix d'un schéma adapté nécessite des informations qualitatives sur la solution.
- L'étude de la stabilité met en évidence l'influence de l'accumulation des erreurs d'arrondi :
- Pour atteindre une précision voulue avec un schéma d'ordre insuffisant, augmenter N est **moins efficace** que choisir un schéma d'ordre plus élevé.

Stabilité, équations raides

- Une équation raide est une équation différentielle pour laquelle certaines composantes de la solution évoluent beaucoup plus rapidement que d'autres.
  - Ceci rend les méthodes numériques instables sauf si le pas est très petit.
- Une équation raide impose de très petits pas pour rester stable avec les méthodes explicites, alors que les méthodes implicites restent stables même avec des grands pas.

Quand une équation est-elle raide ?

- L'équation contient des coefficients très grands (ex : $-1000x$, -10^6x...)
- La solution a plusieurs échelles de temps (ex : variation rapide au début, lente ensuite)
- Les méthodes explicites explosent si tu prends un pas trop grand.

Stabilité, équations raides

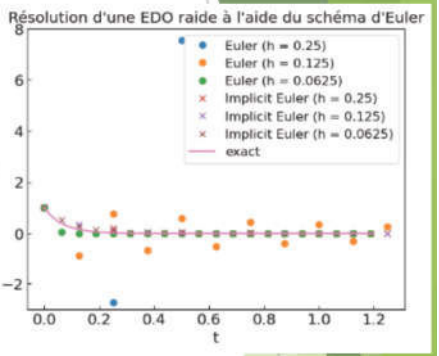
- Équation raide :

$$\frac{dx}{dt} = -15x, \quad x(0) = 1.$$

- Solution exacte :

$$x(t) = e^{-15t} \to 0 \text{ as } t \to +\infty$$

- Méthode explicite d'Euler :
  - Divergence pour $h = 1/4 \to$ Instable.
  - Convergence pour $h = 1/8$, $h = 1/16 \to$ Stable.
- Méthode implicite est plus stable.



Stabilité, équations raides

- Équation raide :

$$\frac{dx}{dt} = -15x, \quad x(0) = 1 \Rightarrow f(x,t) = -15x.$$

- Problème avec Euler explicite :

$$x_{n+1} = x_n + hf(x_n, t_n) = (1 - 15h)x_n$$

$$\Rightarrow \boldsymbol{x_{n+1} = (1 - 15h)^n x_0}$$

$$\Rightarrow \lim_{n \to +\infty} x_{n+1} = \infty \Rightarrow x_{n+1} \textbf{ diverge si } |1 - 15h| > 1$$

- Euler implicite :

$$x_{n+1} = x_n + hf(x_{n+1}, t_{n+1}) = x_n - 15hx_{n+1}$$

$$\Rightarrow x_{n+1} = \frac{x_n}{1 + 15h}$$

$$\Rightarrow \boldsymbol{x_{n+1} = \left(\frac{1}{1 + 15h}\right)^n x_0}$$

$$\Rightarrow \lim_{n \to +\infty} x_{n+1} = 0 \Rightarrow x_{n+1} \textbf{ stable pour tout } \boldsymbol{h > 0}.$$

Systèmes d'équations différentielles ordinaires(EDO)

- On écrit un système d'EDO du premier ordre comme :

$$\begin{cases} \dfrac{dx_1}{dt} = f_1(t, x_1, x_2, \dots, x_n) \\ \dfrac{dx_2}{dt} = f_2(t, x_1, x_2, \dots, x_n) \\ \vdots \\ \dfrac{dx_n}{dt} = f_n(t, x_1, x_2, \dots, x_n) \end{cases}$$ avec des conditions initiales $x_i(t_0) = x_{i,0}$

  - $x_1, x_2, \dots, x_n$ sont les **inconnues** (fonctions du temps).
  - $f_1, f_2, \dots, f_n$ décrivent l'évolution de chaque variable.
- Notation vectorielle : $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$.
- Toutes les méthodes que nous avons abordées ont la même structure lorsqu'elles sont appliquées à des systèmes d'EDO.
- Il suffit de les appliquer composante par composante.

Systèmes d'équations différentielles ordinaires(EDO)

- **Méthode d'Euler :**

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h.\mathbf{f}(\mathbf{x}(t), t).$$

- **RK2 :**

$$\mathbf{k}_1 = h \cdot \mathbf{f}(\mathbf{x}(t), t)$$
$$\mathbf{k}_2 = h \cdot \mathbf{f}\left(\mathbf{x}(t) + \frac{1}{2}\mathbf{k}_1,\ t + \frac{h}{2}\right)$$
$$\mathbf{x}(t+h) = \mathbf{x}(t) + \mathbf{k}_2$$

- **RK4 :**

$$\mathbf{k}_1 = h \cdot \mathbf{f}(\mathbf{x}(t), t)$$
$$\mathbf{k}_2 = h \cdot \mathbf{f}\left(\mathbf{x}(t) + \frac{1}{2}\mathbf{k}_1,\ t + \frac{h}{2}\right)$$
$$\mathbf{k}_3 = h \cdot \mathbf{f}\left(\mathbf{x}(t) + \frac{1}{2}\mathbf{k}_2,\ t + \frac{h}{2}\right)$$
$$\mathbf{k}_4 = h \cdot \mathbf{f}(\mathbf{x}(t) + \mathbf{k}_3,\ t + h)$$
$$\mathbf{x}(t+h) = \mathbf{x}(t) + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

$\mathbf{x}(t)$ est le vecteur des inconnues du système.

$\mathbf{f}(\mathbf{x}, t)$ est le vecteur des dérivées définissant le système.

h est le pas de temps.

Systèmes d'équations différentielles ordinaires(EDO)

Implémentation multidimensionnelle:

Listing: (Python) la méthode d'Euler.

```
def ode_euler_multi(f, x0, t0, h, nsteps):
    """Version multidimensionnelle de la méthode d'Euler.
    """

    t = np.zeros(nsteps + 1)  # Tableau des temps
    x = np.zeros((len(t), len(x0)))  # Tableau des solutions vectorielles
    t[0] = t0
    x[0,:] = x0
    for i in range(0, nsteps):
        t[i + 1] = t[i] + h
        x[i + 1,:] = ode_euler_step(f, x[i], t[i], h)
    return t, x
```

Listing: (Python) la méthode de Runge-Kutta d'ordre 2 (RK2).

```
def ode_rk2_multi(f, x0, t0, h, nsteps):
    """Version multidimensionnelle de la méthode de Runge-Kutta d'ordre
2 (RK2).
    """

    t = np.zeros(nsteps + 1)  # Tableau des temps
    x = np.zeros((len(t), len(x0)))  # Tableau des solutions vectorielles
    t[0] = t0
    x[0,:] = x0
    for i in range(0, nsteps):
        t[i + 1] = t[i] + h
        x[i + 1] = ode_rk2_step(f, x[i], t[i], h)
    return t, x
```

Listing: (Python) la méthode de Runge-Kutta d'ordre 4 (RK4).

```
def ode_rk4_multi(f, x0, t0, h, nsteps):
    """Version multidimensionnelle de la méthode de Runge-Kutta d'ordre
4 (RK4).
    """

    t = np.zeros(nsteps + 1)  # Tableau des temps
    x = np.zeros((len(t), len(x0)))  # Tableau des solutions vectorielles
    t[0] = t0
    x[0] = x0
    for i in range(0, nsteps):
        t[i + 1] = t[i] + h
        x[i + 1] = ode_rk4_step(f, x[i], t[i], h)
    return t, x
```

Systèmes d'équations différentielles ordinaires(EDO)

Exemple:

$$\begin{cases} \dfrac{dx}{dt} = xy - x \\ \dfrac{dy}{dt} = y - xy + \sin^2(t) \end{cases}$$

- C'est un **système non linéaire** d'EDO couplées en $x(t)$ et $y(t)$, dépendant également du temps t via le terme $\sin^2(t)$.

- Écriture sous forme vectorielle

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t).$$

  - le vecteur $\mathbf{x}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$
  - et la fonction : $\mathbf{f}(t, \mathbf{x}) = \begin{bmatrix} xy - x \\ y - xy + \sin^2(t) \end{bmatrix}$

Systèmes d'équations différentielles ordinaires(EDO)

Listing: (Python) Example Extrait du script complet.

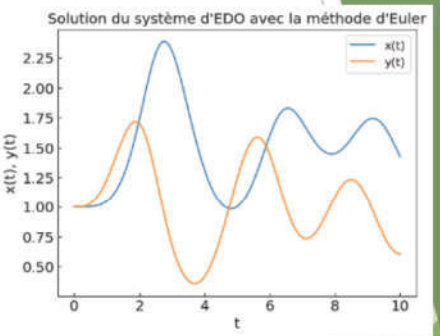
```
# Définition du système d'équations différentielles
def ff(xin, t):
    x = xin[0]
    y = xin[1]
    return np.array([
        x * y - x,            # dx/dt = xy - x
        y - x * y + np.sin(t)**2  # dy/dt = y - xy + sin²(t)
    ])

# Définir l'intervalle temporel
a = 0.          # Temps initial
b = 10.0        # Temps final
N = 100         # Nombre de pas
h = (b - a) / N  # Taille du pas

# Générer les points de temps
tpoints = np.arange(a, b, h)
xpoints = []    # Liste pour stocker x(t)
ypoints = []    # Liste pour stocker y(t)

# Résolution du système avec la méthode d'Euler
sol = ode_euler_multi(ff, [1.0, 1.0], a, h, N)
tpoints = sol[0]            # Liste des temps
xpoints = sol[1][:, 0]      # Composante x de la solution
ypoints = sol[1][:, 1]      # Composante y de la solution

# Tracer les résultats
plt.title("Solution du système d'EDO avec la méthode d'Euler")
plt.xlabel('t')
plt.ylabel('x(t), y(t)')
plt.plot(tpoints, xpoints, label='x(t)')
plt.plot(tpoints, ypoints, label='y(t)')
plt.legend()
plt.show()
```



Systèmes d'EDO du second ordre

- Les équations de Newton/Lagrange sont des systèmes EDOs du second ordre :

$$m_i \frac{d^2 x_i}{dt^2} = F_i(\{x_j\}, \{dx_j/dt\}, t).$$

- Un système de N EDOs du second ordre :

$$\frac{d^2 \mathbf{x}}{dt^2} = \mathbf{f}(\mathbf{x}, d\mathbf{x}/dt, t),$$

- peut être réécrit comme un système de $2N$ EDOs du premier ordre en posant :

$$\begin{cases} \dfrac{d\mathbf{x}}{dt} = \mathbf{v}(t) & \text{(définition de la vitesse)} \\ \dfrac{d\mathbf{v}}{dt} = \mathbf{f}(\mathbf{x}, \mathbf{v}, t) & \text{(loi fondamentale de la dynamique)} \end{cases}$$

- Les méthodes standard (Euler, RK, etc.) s'appliquent alors pour résoudre $\mathbf{x}(t)$ et $\mathbf{v}(t)$.

Systèmes d'EDO du second ordre

Exercice : Le problème de Cauchy associé au modèle électrique

- Déterminer le problème de Cauchy associé au modèle suivant:

$$LCv''(t) + \left(\frac{L}{R_2} + R_1 C\right) v'(t) + \left(\frac{R_1}{R_2} + 1\right) v(t) = e, \quad t \in]0,100]$$

avec les conditions initiales : $v(0) = 0, \quad v'(0) = 0$

Réduction en système d'EDO du premier ordre :

- Posons $x_1(t) = v(t), \; x_2(t) = v'(t) \Rightarrow \begin{cases} x_1'(t) = x_2(t) \\ x_2'(t) = \frac{e}{LC} - \left(\frac{1}{R_2 C} + \frac{R_1}{L}\right) x_2(t) - \left(\frac{R_1}{R_2 LC} + \frac{1}{LC}\right) x_1(t). \end{cases}$

$$\Rightarrow \frac{d}{dt}\boldsymbol{x}(t) = \frac{d}{dt}\begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} = \begin{pmatrix} x_2(t) \\ \frac{e}{LC} - \left(\frac{1}{R_2 C} + \frac{R_1}{L}\right) x_2(t) - \left(\frac{R_1}{R_2 LC} + \frac{1}{LC}\right) x_1(t) \end{pmatrix} = \boldsymbol{f}(t, \boldsymbol{x}(t))$$

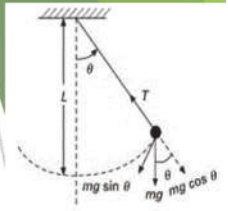
Problème de Cauchy associé :

- Trouver la fonction $x:]0,100] \to \mathbb{R}^2$ vérifiant $\begin{cases} \boldsymbol{x}'(t) = \boldsymbol{f}(t, \boldsymbol{x}(t)), & \forall\, t \in]0,100] \\ \boldsymbol{x}(0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \in \mathbb{R}^2 \end{cases}$

avec $\boldsymbol{f}:]0,100] \times \mathbb{R}^2 \to \mathbb{R}^2$

$$(t, \boldsymbol{x}) \to \begin{pmatrix} x_2(t) \\ \frac{e}{LC} - \left(\frac{1}{R_2 C} + \frac{R_1}{L}\right) x_2(t) - \left(\frac{R_1}{R_2 LC} + \frac{1}{LC}\right) x_1(t) \end{pmatrix}$$

Systèmes d'EDO du second ordre



Exercice : Le problème de Cauchy associé au pendule

- Déterminer le problème de Cauchy associé au modèle du pendule:

$$mL\frac{d^2\theta}{dt^2} = -mg\sin\theta, t \in]0,T] \text{ avec } \theta(0) = \theta_0, \quad \text{et } \theta'(0) = \omega_0$$

Réduction en système d'EDO du premier ordre :

- Posons : $x_1(t) = \theta(t)$, et $x_2(t) = \theta'(t) = \omega(t)$.

$$\Rightarrow \begin{cases} x_1'(t) = x_2(t) \\ x_2'(t) = -\frac{g}{L}\sin(x_1(t)) \end{cases} \Rightarrow \frac{d}{dt}x(t) = \frac{d}{dt}\begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} = \begin{pmatrix} x_2(t) \\ -\frac{g}{L}sin(x_1(t)) \end{pmatrix} = f(t, x(t))$$

Problème de Cauchy associé :

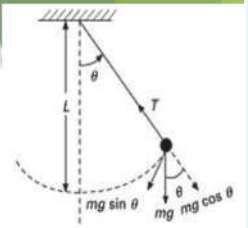
- Trouver la fonction $x:]0,T] \to \mathbb{R}^2$ vérifiant

$$\begin{cases} x'(t) = f(t, x(t)), & \forall\ t \in]0,T] \\ x(0) = \begin{pmatrix} \theta_0 \\ \omega_0 \end{pmatrix} \in \mathbb{R}^2 \end{cases}$$

avec

$$f:]0,T] \times \mathbb{R}^2 \to \mathbb{R}^2$$
$$(t,x) \to \begin{pmatrix} x_2(t) \\ -\frac{g}{L}sin(x_1(t)) \end{pmatrix}$$

Systèmes d'EDO du second ordre



Exemple : Pendule simple

- L'équation du mouvement pour un pendule simple non linéaire :

$$mL\frac{d^2\theta}{dt^2} = -mg\sin\theta.$$

- En posant $\frac{d\theta}{dt} = \omega$, on obtient un système de 2 EDOs du premier ordre :

$$\begin{cases} \frac{d\theta}{dt} = \omega \\ \frac{d\omega}{dt} = -\frac{g}{L}\sin\theta \end{cases} \text{ ou}$$

  - $\theta(t)$ est l'angle de déviation,
  - $\omega(t)$ est la vitesse angulaire,
  - g est l'accélération gravitationnelle,
  - L est la longueur du fil.

- **Approximation linéaire (petits angles) :**

Si $\sin\theta \approx \theta$, la solution analytique est : $\theta(t) \approx \theta_0 \cos\left(\sqrt{\frac{g}{L}}t + \phi\right)$.

Systèmes d'EDO du second ordre

Exemple : Pendule simple

- **Conditions initiales :** Angle initial : $\theta_0 = 20° \approx 0.111\pi$, $L = 0.1\,\text{m}$ et $g = 9.81\,\text{m/s}^2$
  - Comparaison entre la solution numérique (RK4) et l'approximation linéaire.
  - **Régime linéaire :** L'approximation est valable pour θ petit.

Listing: (Python) Example Extrait du script complet.

```
def fpendulum(xin, t):
    theta, omega = xin
    return np.array([omega, -g / L * np.sin(theta)])

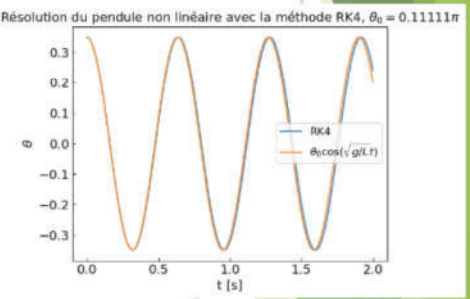
g, L, theta0, omega0 = 9.81, 0.1, (20. / 180.) * np.pi, 0.
a, b, N, h = 0., 2.0, 5000, (2.0 - 0.) / 5000
x0 = np.array([theta0, omega0])

sol = ode_rk4_multi(fpendulum, x0, a, h, N)
tpoints = sol[0]
xpoints = sol[1][:, 0]
ypoints = sol[1][:, 1]

def theta_small_angles(t):
    return theta0 * np.cos(np.sqrt(g / L) * t)

xpoints_small = theta_small_angles(tpoints)

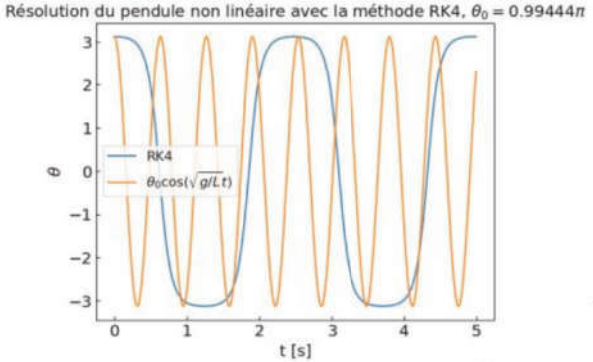
plt.title(f"Résolution du pendule non linéaire avec la méthode RK4,
${{\\theta_0 = {theta0 / np.pi:.5f}\\pi}}$")
plt.xlabel('t [s]')
plt.ylabel('${\\theta}$')
plt.plot(tpoints, xpoints, label='RK4')
plt.plot(tpoints, xpoints_small, label='${\\theta_0 \\cos(\\sqrt{g/L} t)}$')
# plt.plot(tpoints, ypoints, label='y')
plt.legend()
plt.show()
```



Systèmes d'EDO du second ordre

Exemple : Pendule non linéaire (grands angles)

- **Conditions initiales :** Angle initial :$\theta_0 = 179° \approx 0.994\pi$, $L = 0.1\,\text{m}$ et $g = 9.81\,\text{m/s}^2$
- Comparaison entre la solution numérique (RK4) et l'approximation linéaire.
- Pour θ_0 proche de π, l'approximation linéaire échoue.
  - Le pendule exhibe des oscillations non sinusoïdales.



Systèmes d'EDO du second ordre: Pas adaptatif

- Pour une ODE simple, le pas adaptatif est ajusté via :

$$h' = h\left(\frac{30h\delta}{|x_1 - x_2|}\right)^{1/4}.$$

- **Problème :** Comment généraliser $\epsilon = |x_1 - x_2|$ pour un système d'EDOs où $\mathbf{x}$ est un vecteur ?
- **Solutions possibles :**
  - Norme du vecteur différence : $\epsilon = \| \mathbf{x}_1 - \mathbf{x}_2 \|$.
  - Cibler une seule variable (ex. position vs vitesse).
- **Flexibilité :**
  L'implémentation doit permettre de définir ϵ selon le problème physique.

Systèmes d'EDO du second ordre

Implémentation du pas adaptatif:

Listing: (Python) Example Extrait du script complet.

```
# Définition par défaut de l'erreur/distance entre deux vecteurs d'état
# Par défaut : la norme du vecteur différence
def distance_definition_default(x1, x2):
    diff = x1 - x2
    diffnorm = np.sqrt(np.dot(diff, diff))
    return diffnorm

def ode_rk4_adaptive_multi(f, x0, t0, h0, tmax, delta = 1.e-6, distance_definition = distance_definition_default):
    """Résout une EDO dx/dt = f(x,t) de t = t0 à t = t0 + h*steps
    en utilisant la méthode de Runge-Kutta d'ordre 4 avec pas de temps adaptatif.

    Arguments :
      f : fonction définissant l'EDO.
      x0 : valeur initiale de la variable dépendante.
      t0 : valeur initiale du temps.
      h0 : pas de temps initial.
     tmax : temps final.
    delta : précision souhaitée par unité de temps.

    Retour :
    t, x : couple de tableaux correspondant au temps et à la variable dépendante.
    """

    ts = [t0]  # Liste des instants
    xs = [x0]  # Liste des solutions

    h = h0     # Pas de temps courant
    t = t0     # Temps courant
    i = 0      # Indice d'itération
```

Listing: (Python) Example Extrait du script complet (suite).

```
    while t < tmax:
        if t + h >= tmax:
            ts.append(tmax)
            h = tmax - t
            xs.append(ode_rk4_step(f, xs[i], ts[i], h))
            t = tmax
            break

        # Intégration sur deux pas successifs de taille h
        x1 = ode_rk4_step(f, xs[i], ts[i], h)
        x1 = ode_rk4_step(f, x1, ts[i] + h, h)

        # Intégration sur un seul pas de taille 2h
        x2 = ode_rk4_step(f, xs[i], ts[i], 2*h)

        # Calcul de la norme de l'erreur
        diffnorm = distance_definition(x1, x2)

        if diffnorm == 0.:  # Éviter la division par zéro
            rho = 2.**4
        else:
            rho = 30. * h * delta / diffnorm

        if rho < 1.:
            h *= rho**(1/4.)
        else:
            if (t + 2.*h) < tmax:
                xs.append(x1)
                ts.append(t + 2*h)
                t += 2*h
            else:
                xs.append(ode_rk4_step(f, xs[i], ts[i], h))
                ts.append(t + h)
                t += h
            i += 1
            h = min(2.*h, h * rho**(1/4.))

    return ts, xs
```

Systèmes d'EDO du second ordre

Implémentation du pas adaptatif:

Listing: (Python) Example Extrait du script complet.

```
# Définition de l'erreur pour le pendule (en position angulaire
uniquement)
def error_definition_pendulum(x1, x2):
    return np.abs(x1[0] - x2[0])

# Constantes physiques
g, L = 9.81, 0.1

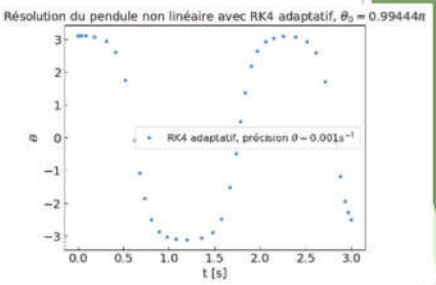
# Conditions initiales
theta0, omega0= (179./180.) * np.pi, 0.

x0 = np.array([theta0, omega0])
a = 0.          # temps initial
b = 3.0         # temps final
N = 500         # nombre de pas initiaux (non utilisés directement)
h0 = (b-a)/N    # pas de temps initial
eps = 1.e-3     # précision sur l'angle theta

# Résolution de l'équation différentielle
sol = ode_rk4_adaptive_multi(fpendulum, x0, a, h0, b, eps,
error_definition_pendulum)

# Extraction des résultats
tpoints = [t for t in sol[0]]
xpoints = [x[0] for x in sol[1]]
ypoints = [x[1] for x in sol[1]]

# Affichage de l'évolution de theta(t)
plt.title(f"Résolution du pendule non linéaire avec RK4 adaptatif,
$\\theta_0 = {theta0 / np.pi:.5f}\\pi$")
plt.xlabel('t [s]')
plt.ylabel('${\\theta}$')
plt.plot(tpoints, xpoints, '.', label='RK4 adaptatif, précision
${\\theta} = ' + str(eps) + ' {s^{-1}}$')
plt.legend()
plt.show()
```



- Le mouvement ne suit pas une sinusoïde parfaite, contrairement au pendule linéarisé.
- On observe des oscillations asymétriques et anharmoniques, typiques d'un pendule non-linéaire à grande amplitude.
- Le schéma adaptatif permet un calcul précis tout en limitant le nombre de points inutiles.

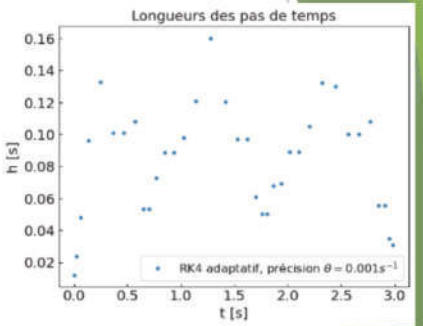
Systèmes d'EDO du second ordre

Implémentation du pas adaptatif:

Listing: (Python) Example Extrait du script complet. (Suite)

```
# Affichage de la taille des pas de temps utilisés
thpoints = []
hpoints = []
for i in range(len(tpoints) - 1):
    thpoints.append(0.5 * (tpoints[i] + tpoints[i+1]))
    hpoints.append(tpoints[i+1] - tpoints[i])

plt.title("Longueurs des pas de temps")
plt.xlabel('t [s]')
plt.ylabel('h [s]')
plt.plot(thpoints, hpoints, '.', label='RK4 adaptatif, précision
${\\theta} = ' + str(eps) + ' {s^{-1}}$')
plt.legend()
plt.show()
```



- h indique la taille du pas entre deux instants.
- Le pas de temps devient plus petit lorsque le pendule est proche de ses positions extrêmes (où la vitesse angulaire change rapidement).
- Il devient plus grand lorsque le mouvement est plus.
- Ce comportement valide l'efficacité du schéma adaptatif, qui ajuste dynamiquement la résolution temporelle en fonction de la complexité du mouvement